Основы языка C

Повторение: сборка программ

- Какие стадии проходит программа от исходного кода до запуска?

- Зачем нужна каждая стадия?

- Что является входом и выходом каждой стадии?

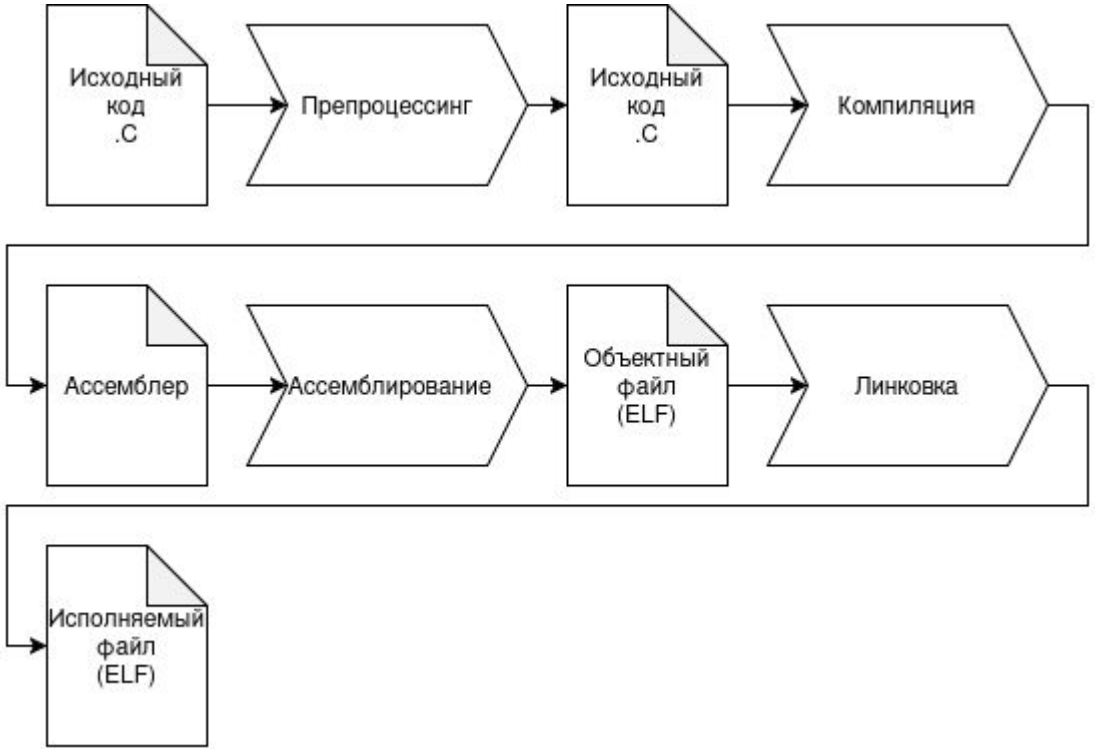
Этапы сборки

- **Препроцессинг**
  Директивы #include и #define.
  Исходный текст .c -> текст .i

- **Компиляция**
  Текст .i -> ассемблер .s

- **Ассемблирование**
  Ассемблер .s -> объектный .o

- **Линковка**
  Объекты и библиотеки -> программа

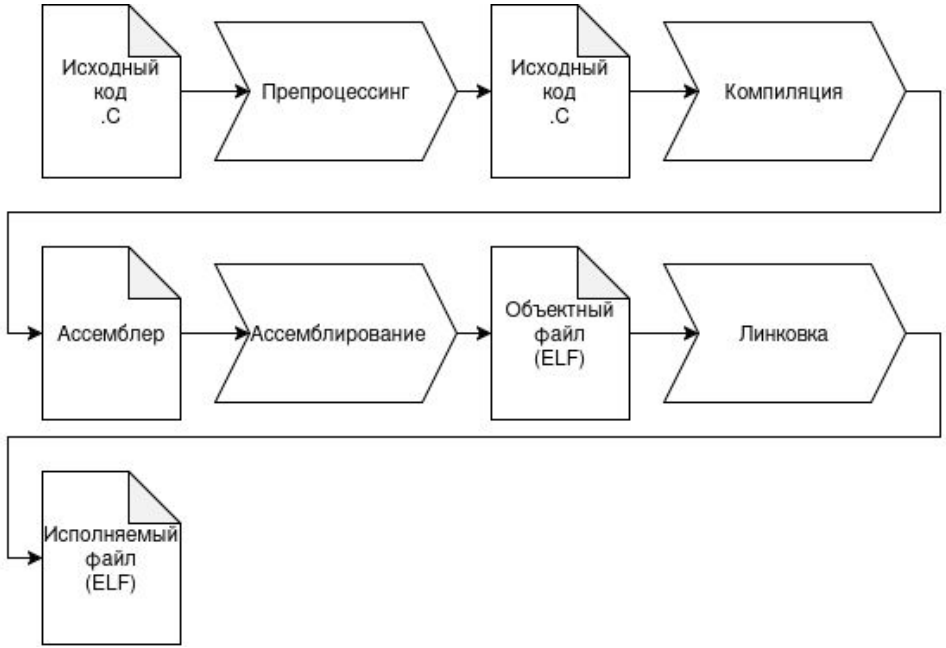


Драйвер компиляции

- gcc и clang принимают файлы и флаги, затем запускают нужные стадии сборки:
  `gcc hello.c -o hello`

- Посмотреть команды драйвера:
  `gcc -### hello.c`

- Остановиться после стадии:
  -E (препроцессинг)
  -S (ассемблер)
  -c (компиляция)



Почему язык C

- Относительная “низкоуровневость” C помогает понять устройство программ и компьютера.

- Проблемы языка C были решены в других языках. Изучая C, мы поймем почему другие языки устроены так.

- Небольшое ядро языка позволяет сосредоточиться на алгоритмах.

- C все еще используется в промышленной разработке.
  Например: ядро Linux, PostgreSQL, CPython, SQLite, OpenSSL, KasperskyOS, Windows.

- Этот курс готовит к C++, ассемблеру и системному программированию

Как появился C

- Задача: писать системные программы и Unix с меньшей зависимостью от ассемблера.
- 1971–1973: Деннис Ритчи разрабатывает C в Bell Labs на основе языка B.
- 1978: книга Кернигана и Ритчи описывает язык до стандартизации.
- 1989: ANSI C.
- 1990: международный стандарт ISO C.

Стандарт языка C

- Стандарт задает синтаксис, семантику и стандартную библиотеку.
- Он нужен для переносимости программ и совместимости реализаций: программист и разработчик компилятора опираются на общие правила.
- у Python нет аналогичного стандарта ISO. Язык описывают официальная документация и PEP. Каждый интерпретатор может вести себя как хочет.
- В курсе используем C99. Флаг компилятора: -std=c99.
- Другие редакции: C11, C17, C23. Некоторые свойства зависят от реализации; для некоторых ситуаций стандарт не задает требований.

Простейшая программа

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- #include <stdio.h>
  дает объявление printf.
- функция main – точка входа в обычной программе.
- return 0 сообщает об успешном завершении.

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Простейшая программа. Можно ли без include

```
int printf(const char*, ...);

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- stdio.h содержит объявление printf.
- Без include можно вручную написать совместимое объявление, как слева. Это объявление, а не определение функции.
- Реализацию printf предоставляет библиотека и находит линкер

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Типы данных

- Тип задает размер, множество значений, представление и допустимые операции.

- Целые: char, short, int, long, long long.
  Для целых есть знаковые (signed) и беззнаковые (unsigned) варианты.

- Вещественные: float, double, long double.

- Логический: в <stdbool.h> – bool, true, false.

- Также есть указатели, массивы, структуры и другие типы.
  О них поговорим на других семинарах.

Размеры типов. Гарантии стандарта

- **sizeof(char) == 1.**
  Байт C содержит CHAR_BIT бит, минимум 8.

- **sizeof(char) ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)**

- int гарантированно представляет хотя бы -32767...32767.
  Типичный размер int – 32 бита, но это **не гарантия стандарта**.

- Проверка размера, максимальных и минимальных значений:
  sizeof, <limits.h>, <float.h>.

- Иногда нужны гарантированные размеры.
  Для этого в <stdint.h> есть int32_t и uint32_t

Типы литералов

- Литерал – число, символ, например 1, 2, ‘a’, “hello”.

- Целый литерал без суффикса получает первый подходящий тип: int, long, long long.

- Суффиксы:
  - U – unsigned,
  - L – long,
  - LL – long long;
  - F – float.

- Иногда литералы необходимы:
  - 1 << 40 – UB (ошибка) – переполнение знакового типа.
  - 1ULL << 40: левый операнд имеет тип unsigned long long.

Неявные и явные преобразования

- Преобразование типов – получение значения одного типа из значения другого по правилам языка.

- Оно нужно для смешанных вычислений, передачи аргументов и записи результата в переменную другого типа.

- Неявное преобразование выполняется автоматически:
  `double x = 5;` или `int + double`.

- Явное приведение задаем записью `(тип)выражение`.
  `(double)a / b` дает вещественное частное двух целых.

- Преобразование может терять данные:
  `(int)3.9` дает `3`

Неявные и явные преобразования

1. Определяются типы операндов и вычисляется выражение.
2. Результат преобразуется к типу переменной.

Явное приведение задает преобразование в указанном месте.

```
double a = 5 / 2;          // 2.0  т.к. типы операндов int
double b = (double)5 / 2;  // 2.5  явное преобразование
int c = 3.9;               // 3    неявное преобразование
```

Операция, выражение, оператор

что здесь является операцией?

что является выражением?

что является оператором?

```
int result = 0;
result = 2 + 3 * 4;
printf("%d\n", result);
```

Операция, выражение, оператор

Операции: =, +, *, вызов printf(...).

Выражения: 2 + 3 * 4 и result = 2 + 3 * 4.

Операторы-выражения: result = 2 + 3 * 4; и printf(...);

```
int result = 0;
result = 2 + 3 * 4;
printf("%d\n", result);
```

Основные операции C

- **Арифметика:**
  `+, -, *, /, %, x++, x–`
  `5 / 2 == 2, -5 / 2 == -2, -5 % 2 == -1`

- **Сравнения:**
  `<, <=, >, >=, ==, !=`

- **Логика:**
  `!, &&, ||`
  0 означает false, любое ненулевое значение – true.

- **Присваивание**:
  `=, +=, -=, *=, /=, %=`

Битовые операции

```
unsigned int x = 10U; // ...1010
unsigned int y = 12U; // ...1100
x & y;                // 8
x | y;                // 14
x ^ y;                // 6
x << 1;               // 20
```

- `&` И
- `|` ИЛИ
- `^` исключающее ИЛИ
- `~` инверсия
- `<<` и `>>` битовые сдвиги

Приоритет и порядок вычислений

Приоритет	Операторы
1	`[]` `()` `.` `->` `++` `--` (постфикс)
2	`sizeof` `(тип)` `&` `*` `+` `-` `~` `!` `++` `--` (префикс)
3	`*` `/` `%`
4	`+` `-`
5	`<<` `>>`
6	`<` `>` `<=` `>=`
7	`==` `!=`
8	`&`
9	`^`
10	`|`
11	`&&`
12	`||`
13	`?` `:`
14	`=` `*=` `/=` `%=` `+=` `-=` `<<=` `>>=` `&=` `^=` `|=`
15	`,`

Приоритет и порядок вычислений

```
2 + 3 * 4           // 14
(2 + 3) * 4         // 20
x & 1 == 0          // x & (1 == 0)
(x & 1) == 0        // проверяем четное ли x
```

Короткая (ленивая) логика: условие if

```
int a = 10;
int b = 0;

if (b != 0 && a / b > 2) {
    puts("Quotient > 2");
} else {
    puts("Condition is false");
}
```

- && сначала вычисляет левый операнд.
- Если b == 0, левый операнд ложен: **деление не выполняется**.
- Если b != 0, вычисляется a / b > 2.
- Для || правый операнд не вычисляется, если левый уже истинен.

Неопределенное поведение: UB

- **Undefined behavior**: стандарт не предъявляет требований к поведению программы в такой ситуации.

- При UB может произойти что угодно: аварийное завершение, неверный результат или неожиданное выполнение кода. **Стандарт ничего не гарантирует.** Ваш компьютер может сгореть. Ваш кот может уронить люстру. Даже если у вас нет кота или люстры.

- Примеры: деление целого на ноль, знаковое переполнение, выход за границы массива.

- Компилятор оптимизирует, исходя из того, что на корректном пути выполнения UB не возникает.

- Успешная компиляция и один удачный запуск не гарантируют корректность программы. UB может “стрельнуть” 1 раз на миллион.

UB и оптимизация: x + 1 > x

- Если x + 1 представимо в int, результат сравнения равен 1.
- При x == INT_MAX сложение вызывает UB – знаковое переполнение
- Clang 18.1 при -O2 заменяет функцию на return 1.
  Если мы передадим x == INT_MAX, получим 1. Это как минимум странно.

```
int greater(int x) {
    return x + 1 > x;
}
```

https://godbolt.org/z/e46dh4159

Функция, которую никто не вызывал

```
#include <stdio.h>

int main() {
  unsigned int x = 0;
  while (x >= 0) {
    x = x + 1;
  }
}

void unreachable_function() {
  printf("You would never see this!\n");
}
```

- Цикл не завершится:
  x – беззнаковый.
- У цикла нет наблюдаемых побочных эффектов. Это UB.
- Что покажет программа при -O0 и -Ofast?

https://godbolt.org/z/x3G3aYrEx

Поведение, определяемое реализацией

```
int char_is_signed(void) {
    char c = (char)-1;
    return c < 0;
}
```

- Implementation-defined behavior (IDB): стандарт допускает несколько вариантов и требует документировать выбор реализации.

- GCC 14.1, -fsigned-char: вывод 1
- GCC 14.1, -funsigned-char: вывод 0

- Различаться могут компилятор, платформа и флаги сборки.

- В этих двух конфигурациях программа корректна.

https://godbolt.org/z/sdj39YxGz

Виды поведения программы

- Определенное поведение
  5 / 2 дает 2. Реализация обязана соблюдать правило.

- Implementation-defined
  Знаковость char выбирает и документирует реализация.

- Unspecified
  Порядок вычисления f() и g() в f() + g() не задан.
  Допустимый выбор не требуется документировать.

- Undefined
  1 / 0: стандарт не предъявляет требований.

Литература: язык C

- Б. Керниган, Д. Ритчи. «Язык программирования C», 2-е издание.
- C. Прата. C Primer Plus
- ISO C99 + TC1–TC3, проект WG14 N1256.
  Будет доступен на контрольных как **единственный справочник**, нужно научиться с ним работать.

Драфт N1256

Литература: алгоритмы

- А. Бхаргава. «Грокаем алгоритмы».
- Т. Рафгарден. Algorithms Illuminated, части 1–4.
- Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. «Алгоритмы. Построение и анализ».